AGENTGATE: PRETRAINING AND BENCHMARKING A SMALL LANGUAGE MODEL WITH AN INVESTIGATION OF ITS INDUCTION CIRCUITS

Mohammad Sydul Hasan
Chittagong University of Engineering and Technology
sydul.tawsif@gmail.com

ABSTRACT

Interest in small language models (SLMs) has grown because of their potential to reduce memory requirements and inference costs. Smaller models may be useful as components in AI agents, edge applications, and resource constrained systems. Motivated by this direction, we built AgentGate, a 142.4-million-parameter language model trained entirely from scratch on a mixture of FineWeb-Edu-Dedup (80%) and Cosmopedia-v2 (20%). Our goal was not only to develop a working pretrained model, but to explore what could be achieved with a very limited compute budget. We trained on an NVIDIA L4 GPU through Modal, targeting its $30 monthly compute credit. The final training run consumed approximately 36 GPU-hours and cost $28.79 in GPU usage; total experimental expenditure was approximately $34, including CPU charges, smoke tests, and checkpoint restarts. Beyond training, we benchmarked AgentGate against GPT-2 Small, Pythia-160M, and SmolLM-135M on ARC-Easy, HellaSwag, and PIQA. We then investigated induction-like circuits through sequence repetition tests, attention head specialization analysis, causal head ablations, and experiments on cross layer functional dependencies.

Table of Contents

1 Choosing the Model Scale **4**
1.1 GPU Hour Budget . . . 4
1.2 Estimating Usable Compute . . . 4
1.3 Applying Chinchilla . . . 4
1.4 Fixing the Architecture . . . 4

2 Tokenizer Design **5**
2.1 Selecting the Vocabulary Size . . . 5
2.2 Initial Tokenizer Experiment . . . 5
2.3 Larger Scale Vocabulary Evaluation . . . 5

3 Pretraining Data Preparation **6**
3.1 Train–Validation Separation . . . 6
3.2 Streaming and Shuffling . . . 6
3.3 Document Boundaries . . . 7
3.4 Binary Storage and Sharding . . . 7
3.5 Full Corpus Token Audit . . . 7

4 Model Architecture **8**
4.1 Decoder-Only Design . . . 8
4.2 Depth and Hidden Dimension . . . 8
4.3 Multi-Head Self-Attention . . . 10
4.4 Rotary Positional Embeddings . . . 10
4.5 Context Length . . . 10
4.6 Pre-RMSNorm . . . 10
4.7 SwiGLU Feed-Forward Network . . . 11
4.8 Other Architectural Choices . . . 11

5 Results of the Pretrained Base Model **12**
5.1 Training and Validation Loss . . . 12
5.2 Learning-Rate Schedule and Optimization Stability . . . 12
5.3 Training Efficiency . . . 13
5.4 Compute Cost . . . 13

6 Zero-Shot Benchmark Evaluation **14**
6.1 Evaluation Setup . . . 14
6.2 Comparison Models . . . 15
6.3 Benchmark Results . . . 15
6.4 Discussion . . . 15

7 Mechanistic Interpretability: Investigating Induction Circuits **16**
7.1 Introduction . . . 16

8 Model and Evaluation Setting **16**

9 Experiment 1: Repeated-sequence behavior **17**
9.1 Question and motivation . . . 17
9.2 Experimental setup . . . 17
9.3 Results and Interpretation . . . 18

10 Experiment 2: Attention-head specialization **19**
10.1 What patterns are we looking for? . . . 19
10.2 How we measured head specialization . . . 20
10.3 What we found . . . 20

11 Experiment 3: Causal Attention-Head Ablation **21**
11.1 Why Are Attention Scores Not Enough? . . . 21
11.2 How We Performed the Interventions . . . 22
11.3 Results and interpretation . . . 23

12 Experiment 4: Functional Dependency Between Earlier and Later Heads **24**
12.1 The Question Left by Ablation . . . 24
12.2 How We Tested the Dependency . . . 25
12.3 Results and Interpretation . . . 25

1 Choosing the Model Scale

Before designing the architecture, we first needed to decide how large a model we could realistically pretrain with our available compute budget. Training a model that was too large on too few tokens would leave it undertrained, while choosing a very small model would not make good use of the available data. We therefore derived the model size and token budget from the compute budget, using the Chinchilla scaling law results [1] and standard heuristics.

1.1 GPU Hour Budget

We pretrained on an NVIDIA L4 GPU rented through Modal [2], budgeted at approximately

$$\$0.80 \text{ per GPU hour.} \tag{1}$$

Our Modal account provided approximately \$30 of monthly compute credit, so this was the practical upper bound for the run. Ignoring small non-GPU costs,

$$\frac{\$30}{\$0.80/\text{hour}} \approx 37.5 \text{ GPU-hours.} \tag{2}$$

We did not plan to spend all of it on the main run. We kept roughly 5 GPU-hours (about \$4) in reserve for validation, checkpointing, startup overhead, and debugging, which left about

$$T \approx 32 \text{ GPU-hours} = 32 \times 3600 = 115{,}200 \text{ s} \tag{3}$$

for pretraining.

1.2 Estimating Usable Compute

The L4's peak dense BF16 throughput is 121 TFLOPS, but real training does not reach the peak. So, we assumed a Model FLOPs Utilization (MFU) about 16%, giving an effective compute rate of

$$0.16 \times 121 \times 10^{12} \approx 1.9 \times 10^{13} \text{ FLOPs/s.} \tag{4}$$

The total training compute available is then

$$C \approx 1.9 \times 10^{13} \times 115{,}200 \approx 2.2 \times 10^{18} \text{ FLOPs.} \tag{5}$$

1.3 Applying Chinchilla

Chinchilla models training compute as

$$C \approx 6ND, \tag{6}$$

where N is the parameter count and D the number of training tokens, and finds a compute optimal ratio of roughly

$$D \approx 20N. \tag{7}$$

Substituting $D = 20N$ gives $C \approx 120N^2$, so

$$N \approx \sqrt{\frac{C}{120}} = \sqrt{\frac{2.2 \times 10^{18}}{120}} \approx 1.36 \times 10^8 \approx 135\text{M}, \qquad D \approx 20N \approx 2.7\text{B}. \tag{8}$$

1.4 Fixing the Architecture

We then chose architectural dimensions (number of Transformer layers, hidden size, attention heads, and feed-forward width) close to this target. The resulting architecture contains

$$\boxed{142{,}408{,}320 \text{ parameters}}. \tag{9}$$

Keeping the token budget at 2.7B, the token-to-parameter ratio is

$$\frac{2.7\text{B}}{142.4\text{M}} \approx 18.96 \approx \boxed{19 \text{ tokens per parameter}}, \tag{10}$$

close to the Chinchilla guideline.

It is important to note that Chinchilla scaling was used here as a design heuristic, not as a claim that 142M parameters is mathematically optimal for exactly 2.7B tokens. Our goal was choosing a model large enough to be useful, provide it with an appropriate amount of training data, and ensure that the entire pretraining run could actually be completed within our available compute budget.

2 Tokenizer Design

Before pretraining the model, we needed a tokenizer that could represent the corpus efficiently without allocating too much of the model's capacity to the vocabulary. We chose Byte-Level Byte Pair Encoding (BPE).

BPE starts from small units and repeatedly merges frequently occurring token pairs. As a result, common words and character sequences can be represented using relatively few tokens, while uncommon words can still be decomposed into smaller pieces.

The tokenizer was implemented using the Hugging Face tokenizers library. We used a ByteLevel pre-tokenizer and decoder, with `<|endoftext|>` as the document boundary special token.

2.1 Selecting the Vocabulary Size

Vocabulary size introduces a direct trade-off. A vocabulary that is too small tends to fragment text into more tokens. This increases sequence lengths and therefore increases the amount of computation required to process the same amount of text. A very large vocabulary has the opposite problem. It improves compression, but it also increases the size of the embedding matrix and may create a large tail of tokens that occur only rarely during training. Extremely rare or poorly trained tokens can end up with weakly learned embeddings and occasionally produce unexpected model behavior. A well known example is the historical SolidGoldMagikarp [3] token, which became associated with so called glitch tokens. This does not mean that every rare token is problematic, but it is another reason to avoid unnecessarily large vocabularies and to check token frequency coverage before pretraining. For our model, this trade-off was particularly important because the token embedding matrix scales as:

$$V \times d_{\text{model}} \tag{11}$$

where V is the vocabulary size and d_{model} is the hidden dimension. We therefore compared two practical candidates:

$$24{,}000 \quad \text{and} \quad 32{,}000 \tag{12}$$

2.2 Initial Tokenizer Experiment

We first trained both tokenizers using 10,000 training documents and evaluated them on 2,000 held-out documents.

Table 1: Initial Tokenizer Experiment

Metric	24K	32K
Chars / Token ↑	4.492	4.605
Unseen vocabulary entries ↓	726	1,767
Entries seen 1–9 times ↓	6,041	12,680

The 32K tokenizer achieved roughly 2.5% better compression, meaning that the same text could be represented with fewer tokens.

However, it also appeared to have a much larger low frequency tail. Based only on this experiment, the 24K vocabulary seemed safer.

We did not immediately accept that conclusion. The tokenizer had only seen 10,000 training documents, and the evaluation corpus contained only 2,000 documents. A larger vocabulary naturally requires more data before its learned tokens receive sufficient coverage. The poor rare token statistics of the 32K tokenizer could therefore have been an artifact of the small experiment rather than a weakness of the vocabulary size itself.

2.3 Larger Scale Vocabulary Evaluation

We repeated the comparison using substantially more data:

- 100,000 tokenizer-training documents
- 20,000 held-out evaluation documents
- identical dataset mixture and random seed for both tokenizers
- evaluation divided into 10 blocks of 2,000 documents

The block based evaluation also allowed us to check whether the compression advantage was consistent across different parts of the corpus.

The results were:

Table 2: Larger-Scale Vocabulary Evaluation

Metric	24K	32K
Evaluation tokens ↓	19,289,316	18,802,803
Chars / Token ↑	4.5270	4.6442
Unseen vocabulary ↓	0.433%	0.509%
Rare entries, frequency 1–9 ↓	1.125%	1.759%
Block mean chars / token ↑	4.5285	4.6457
Block standard deviation ↓	0.0316	0.0316

The 32K vocabulary retained an approximately 2.6% compression advantage. The same evaluation corpus required about 2.5% fewer tokens than with the 24K tokenizer. At the same time, the difference in rare and unseen vocabulary entries became much smaller than in the initial experiment. This indicated that the earlier result had been strongly influenced by insufficient tokenizer training and evaluation coverage. The almost identical block standard deviations were also useful. They showed that the compression advantage of the 32K tokenizer was stable across the evaluation corpus rather than being produced by a small subset of documents. Based on these experiments, we selected:

$$\boxed{V = 32{,}000} \tag{13}$$

The 32K vocabulary provided a consistent improvement in compression while keeping the low n frequency tail at an acceptable level.

3 Pretraining Data Preparation

The final tokenizer was trained on 100,000 documents sampled from the same underlying distribution used for language model pretraining. Once the tokenizer was finalized, it was frozen and used for the entire corpus.

Our pretraining mixture contained two sources:

- FineWeb-Edu-Dedup: approximately 80%
- Cosmopedia-v2: approximately 20%

The mixture ratio was controlled by token count rather than document count. This matters because documents from the two sources can have very different lengths. The prepared dataset contained approximately:

$$\boxed{2.700\text{B training tokens}} \tag{14}$$

and:

$$\boxed{10.006\text{M validation tokens}} \tag{15}$$

3.1 Train–Validation Separation

Validation data was prepared before the training portion. After collecting the required validation documents from each streaming source, the same source iterators continued forward when constructing the training corpus. This ensured that documents already assigned to validation were not subsequently reused for language model training.

3.2 Streaming and Shuffling

The source datasets were processed in streaming mode. Loading the entire corpus into memory simply to perform a global shuffle would have been impractical at this scale. Each source used a streaming shuffle buffer of 10,000 documents. This was useful because documents in large web corpora may have strong local ordering. For example, before shuffling, part of the stream might look like:

$$\text{football} \to \text{basketball} \to \text{baseball} \to \text{cooking} \to \text{recipes} \to \text{restaurants} \tag{16}$$

Here, several sports documents are followed by several food related documents. Training directly on such ordering would expose the model to long local runs of similar content. With buffer based shuffling, the same region might instead appear as:

$$\text{football} \rightarrow \text{recipes} \rightarrow \text{basketball} \rightarrow \text{restaurants} \rightarrow \text{baseball} \rightarrow \text{cooking} \tag{17}$$

This reduces local correlations and gives the model a more mixed sequence of documents. It is not equivalent to a perfect global shuffle, but it provides a useful compromise between randomness and memory efficiency. The two source streams were shuffled separately before being combined. During corpus construction, we tracked their accumulated token counts and selected documents in a way that kept the final mixture close to the target 80/20 token ratio.

3.3 Document Boundaries

Documents were preserved as complete documents during tokenization rather than being manually truncated to a fixed sequence length. An `<|endoftext|>` token was inserted after each document:

```
document 1
<|endoftext|>
document 2
<|endoftext|>
document 3
<|endoftext|>
```

During pretraining, fixed-length 2,048-token sequences could then be drawn from this continuous token stream. This allowed us to use essentially every available token without padding individual documents to the model context length.

3.4 Binary Storage and Sharding

Token IDs were stored using `uint16`. This is sufficient because a 32K vocabulary requires IDs only in the range:

$$0 \leq id < 32{,}000 \tag{18}$$

The training corpus was divided into 100M-token binary shards rather than stored in one multi billion token file. Conceptually:

```
train_000.bin     ~100M tokens
train_001.bin     ~100M tokens
train_002.bin     ~100M tokens
...
train_026.bin     ~100M tokens
train_027.bin     remaining tokens
```

Sharding made the corpus easier to inspect, transfer, memory-map, and load during training.

3.5 Full Corpus Token Audit

After preprocessing, we counted the frequency of every vocabulary entry across the complete training corpus. Only:

$$46/32{,}000 \tag{19}$$

tokens were never observed, corresponding to approximately:

$$\boxed{0.14\%} \tag{20}$$

of the vocabulary.

This was lower than the unseen vocabulary rate measured during the earlier tokenizer evaluation. As illustrated in Figure 1, 45 of the 46 zero frequency tokens belonged to the tokenizer's initial byte level alphabet, while only one was a learned BPE token. The unused base symbols were primarily concentrated in byte ranges associated with ASCII control characters and invalid or infrequently occurring UTF-8 byte sequences. This suggests that the zero frequency entries largely reflect the tokenizer's byte-coverage mechanism rather than underutilization of learned subword units.

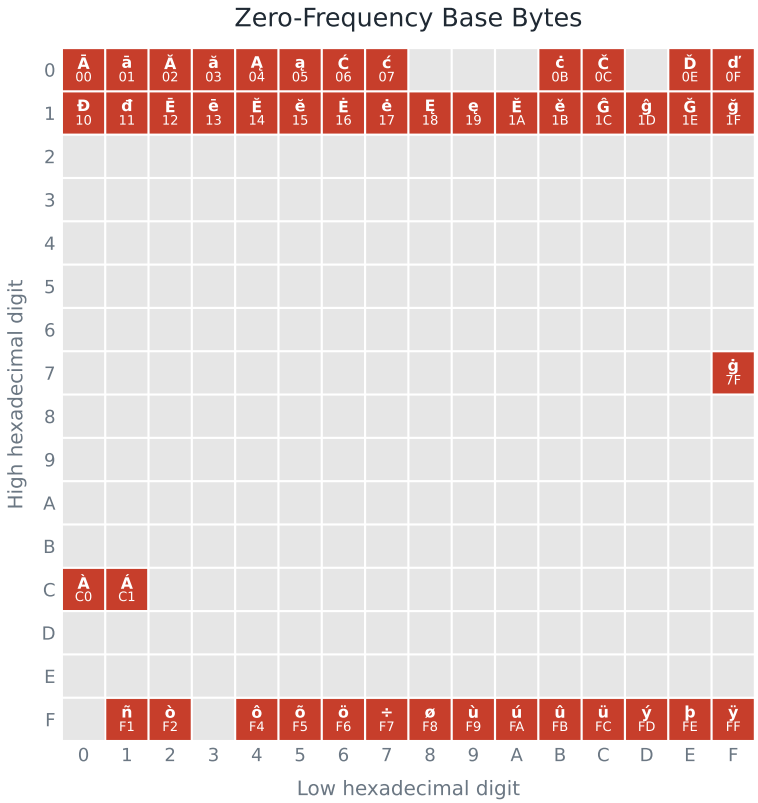


Figure 1: Distribution of zero frequency byte level tokens across the 256-byte alphabet

4 Model Architecture

With the model scale, tokenizer, and corpus fixed, we designed a conventional decoder-only Transformer, illustrated in Figure 2.

4.1 Decoder-Only Design

We chose a decoder-only Transformer because our pretraining objective is standard causal language modeling. Given a sequence

$$x_1, x_2, \ldots, x_T, \tag{21}$$

the model factorizes its probability as

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1}). \tag{22}$$

At each position, the model predicts the next token using only the tokens that came before it:

$$P(x_t \mid x_{<t}). \tag{23}$$

Training minimizes the average next token cross-entropy:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^{T} \log P(x_t \mid x_{<t}). \tag{24}$$

4.2 Depth and Hidden Dimension

We adopt a relatively deep and narrow architecture with

$$L = 24 \tag{25}$$

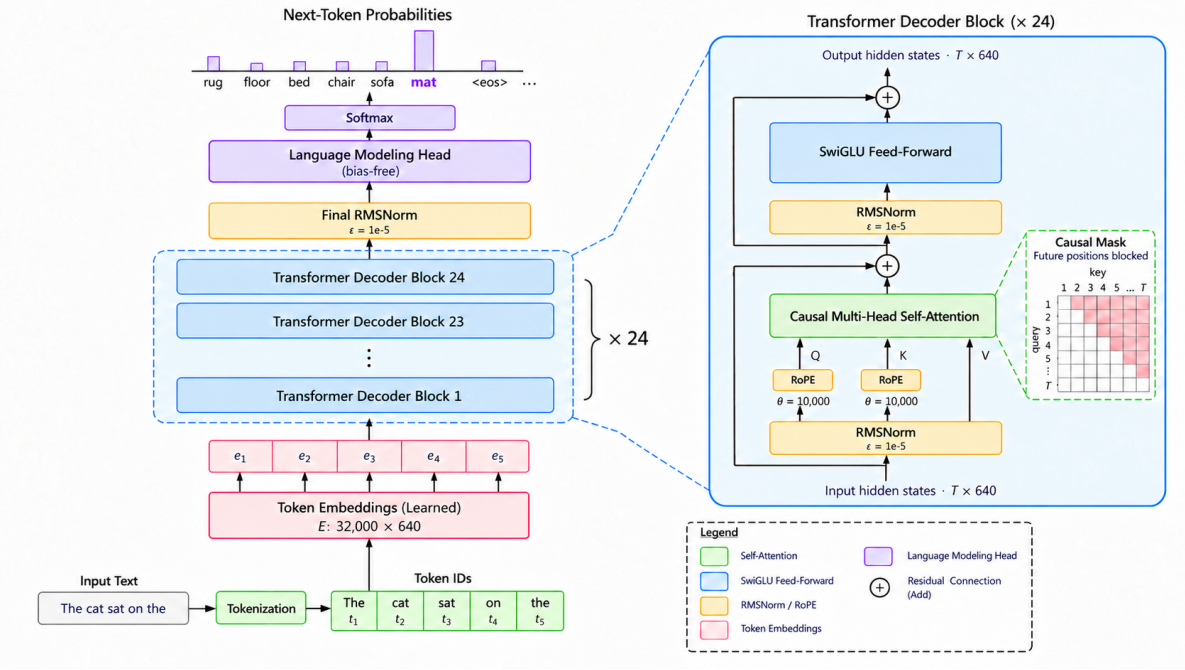


Figure 2: AgentGate Architecture Diagram

Table 3: Model Architecture Specifications

Component	Specification
Architecture	Dense decoder-only Transformer
Parameters	142,408,320
Vocabulary	32,000
Transformer layers	24
Hidden size	640
Attention	Multi-Head Attention
Attention heads	10
Head dimension	64
Context length	2,048
Positional encoding	RoPE
RoPE theta	10,000
Normalization	Pre-RMSNorm
RMSNorm epsilon	1e-5
Feed-forward network	SwiGLU
Intermediate size	1,792
Linear biases	None
Dropout	0.0
Cross-layer sharing	None
Training objective	Next-token cross-entropy

Transformer layers and a hidden dimension of

$$d_{\text{model}} = 640. \tag{26}$$

Prior work, such as [4], has shown that, under constrained parameter budgets, increasing Transformer depth can improve compositional representation and generalization, although the benefits diminish as depth increases. Motivated by these findings, we favored a depth oriented configuration rather than making the model substantially wider and shallower.

4.3 Multi-Head Self-Attention

Each Transformer block uses standard Multi-Head Attention (MHA). For an input representation

$$X \in \mathbb{R}^{T \times d_{\text{model}}}, \tag{27}$$

we compute queries, keys, and values as

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V. \tag{28}$$

Here,

$$W_Q, W_K, W_V \in \mathbb{R}^{640 \times 640}. \tag{29}$$

The 640-dimensional hidden representation is divided across 10 attention heads. Therefore,

$$d_{\text{head}} = \frac{d_{\text{model}}}{H} = \frac{640}{10} = 64. \tag{30}$$

Equivalently,

$$640 = 10 \times 64. \tag{31}$$

Thus, each attention head operates on a 64-dimensional representation.

4.4 Rotary Positional Embeddings

Self-attention by itself does not encode the order of tokens. We therefore use Rotary Positional Embeddings (RoPE) with base

$$\theta = 10{,}000. \tag{32}$$

Unlike learned absolute positional embeddings, RoPE does not require a separate trainable embedding vector for every position. Instead, positional information is incorporated directly into the query and key representations used by self attention. This made RoPE a natural choice for a modern decoder only architecture while adding no learned positional parameters.

4.5 Context Length

We selected a context length of

$$\boxed{\text{2048 tokens}}. \tag{33}$$

A longer context would allow the model to process more information at once, but the computational cost of self attention grows rapidly with sequence length. For our pretraining budget, 2,048 tokens provided a practical compromise between useful context length and computational cost.

4.6 Pre-RMSNorm

Each Transformer block uses pre-normalization:

$$x' = x + \text{Attention}\left(\text{RMSNorm}(x)\right), \tag{34}$$

followed by

$$x'' = x' + \text{FFN}\left(\text{RMSNorm}(x')\right). \tag{35}$$

This differs from post-normalization, where normalization is applied after the residual addition. We adopted Pre-RMSNorm for AgentGate's 24-layer decoder architecture. Previous studies have shown that Post-LN can achieve strong performance in relatively shallow Transformers, but training often becomes unstable as network depth increases due to gradient propagation difficulties [5]. Furthermore, Xiong et al. [6] demonstrated that Post-LN can produce large gradients near the output layers at initialization, making learning rate warmup particularly important for stable optimization. These findings motivate our use of pre-normalization to support stable training in AgentGate's deeper architecture.

4.7 SwiGLU Feed-Forward Network

The feed-forward sublayer uses SwiGLU instead of the original Transformer's ReLU-based MLP. Given hidden state x, we first compute two projections:

$$g = xW_g, \qquad u = xW_u. \tag{36}$$

The gated activation is

$$h = \mathrm{SiLU}(g) \odot u, \tag{37}$$

where

$$\mathrm{SiLU}(z) = z\sigma(z). \tag{38}$$

The result is then projected back to the hidden dimension:

$$\mathrm{FFN}(x) = hW_d. \tag{39}$$

With hidden size

$$d = 640 \tag{40}$$

and intermediate dimension

$$m = 1792, \tag{41}$$

the three projection matrices have shapes

$$W_g \in \mathbb{R}^{640\times 1792}, \tag{42}$$

$$W_u \in \mathbb{R}^{640\times 1792}, \tag{43}$$

and

$$W_d \in \mathbb{R}^{1792\times 640}. \tag{44}$$

The choice of 1792 follows from the parameter structure of SwiGLU. A conventional two-matrix Transformer FFN with an expansion of $4d$ has approximately

$$d(4d) + (4d)d = 8d^2 \tag{45}$$

parameters. SwiGLU instead uses three matrices:

$$dm + dm + md = 3dm. \tag{46}$$

To roughly match the parameter budget of a traditional $4d$ FFN,

$$3dm \approx 8d^2. \tag{47}$$

Cancelling d,

$$3m \approx 8d, \tag{48}$$

which gives

$$m \approx \frac{8}{3}d. \tag{49}$$

For $d = 640$,

$$m \approx \frac{8}{3}(640) = 1706.67. \tag{50}$$

We selected the nearby dimension

$$\boxed{m = 1792}, \tag{51}$$

which satisfies

$$1792 = 7 \times 256 \tag{52}$$

and gives an expansion ratio of

$$\frac{1792}{640} = 2.8. \tag{53}$$

4.8 Other Architectural Choices

We kept several remaining design choices intentionally simple.

No linear biases. The attention and feed-forward linear projections do not use bias terms. Their contribution would be small compared with the matrix parameters, and removing them keeps the architecture consistent with many modern decoder only language models.

No dropout. Dropout was set to 0.0. With billions of training tokens, the model already sees a large and diverse stream of data. We therefore relied primarily on data scale rather than dropout for regularization.

5 Results of the Pretrained Base Model

The pretraining run was completed for 20,599 optimizer steps, corresponding to approximately 2.70B training tokens. Training remained stable throughout the run and aprx. completed within the planned compute budget.

5.1 Training and Validation Loss

Figure 3 shows the training and validation loss throughout pretraining. The loss decreased rapidly during the early stage of training and then improved more gradually as training progressed.

We used two levels of validation during training. A smaller subset validation evaluated approximately 2.048M tokens at regular intervals, allowing us to monitor model quality without repeatedly evaluating the entire validation corpus. At selected training milestones and at the end of pretraining, we performed a full validation over approximately 10.006M tokens.

The final full-validation loss was

$$\boxed{2.8964}, \tag{54}$$

corresponding to a perplexity of

$$\mathrm{PPL} = e^{2.8964} \approx \boxed{18.11}. \tag{55}$$

Both validation measurements followed the same overall downward trend as the training loss. The best full-validation loss was obtained at the final checkpoint, and there was no late increase in validation loss. This suggests that the model continued to improve throughout the allocated token budget without clear evidence of overfitting.

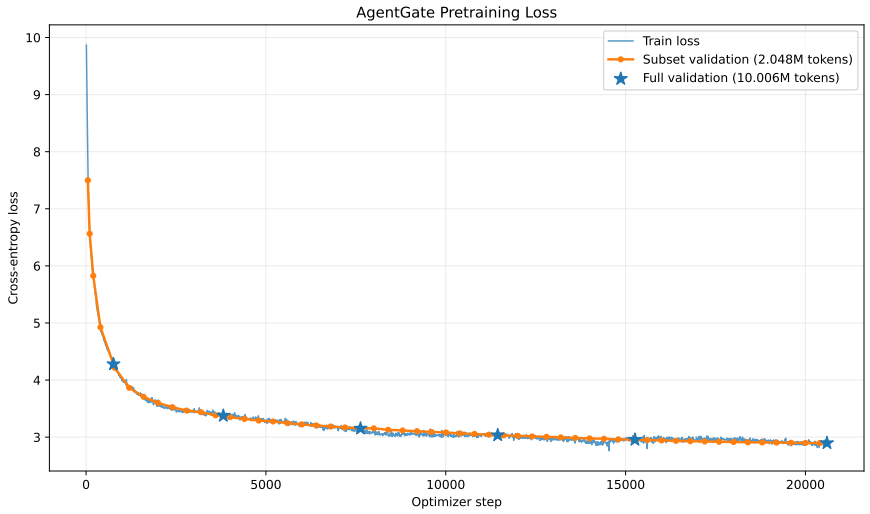


Figure 3: Training loss together with subset validation and full validation.

5.2 Learning-Rate Schedule and Optimization Stability

We used a short linear warmup followed by cosine learning-rate decay. The learning rate reached a maximum of

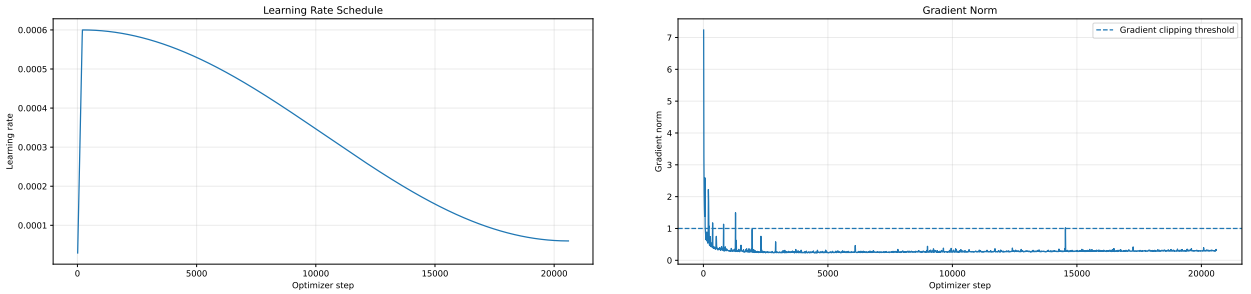
$$6 \times 10^{-4} \tag{56}$$

and gradually decayed to

$$6 \times 10^{-5} \tag{57}$$

by the end of training, as shown in Figure 4a.

Gradient clipping was applied at a norm of 1.0. As shown in Figure 4b, relatively large gradient norms occurred mainly during the earliest training steps. The gradient norm quickly decreased and remained well below the clipping threshold for most of the run. Only occasional spikes were observed later in training. Overall, this indicates stable optimization without persistent gradient instability.



(a) Learning rate schedule used during pretraining.

(b) Gradient norm throughout training.

Figure 4: Optimization dynamics during pretraining.

5.3 Training Efficiency

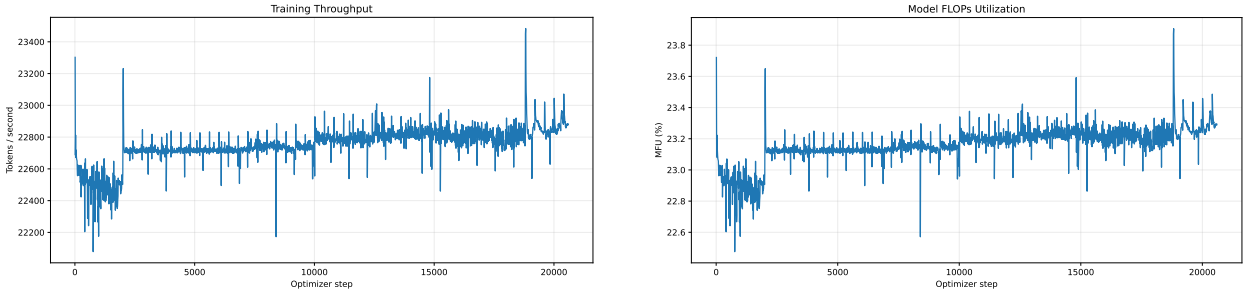
Training throughput remained relatively stable throughout the run, averaging approximately

$$\boxed{22.75\text{K tokens/s}}. \tag{58}$$

Model FLOPs Utilization (MFU) averaged approximately

$$\boxed{23.2\%} \tag{59}$$

on the NVIDIA L4. Both throughput and MFU remained stable after the initial portion of training, suggesting that the training pipeline maintained consistent GPU utilization.



(a) Training throughput measured in tokens per second.

(b) Model FLOPs Utilization during pretraining.

Figure 5: Hardware and computational efficiency metrics during pretraining.

5.4 Compute Cost

The complete run required approximately 36 wall-clock GPU hours. At an L4 price of approximately

$$\$0.80/\text{hour}, \tag{60}$$

the final GPU cost was approximately $28.79. This remained within our original $30 compute budget. The cost increased almost linearly with training progress, as shown in Figure 6. However, the total experimental expenditure reached approximately $34 after accounting for additional costs associated with CPU utilization, preliminary smoke tests, and checkpoint restarts.

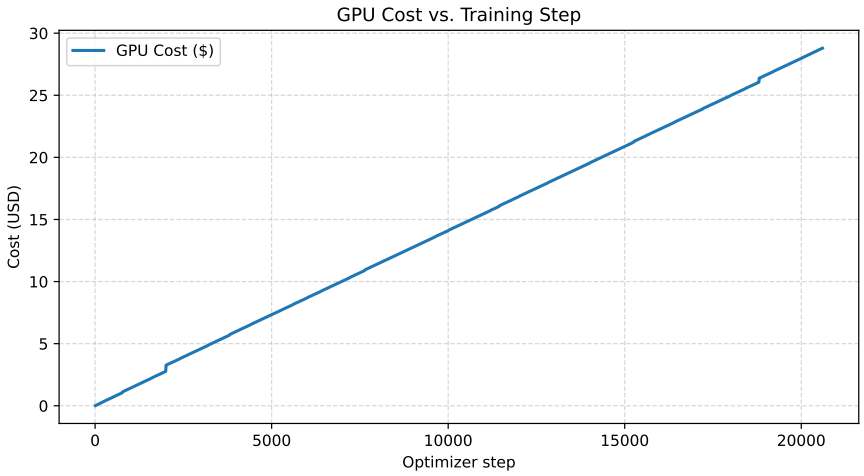


Figure 6: Accumulated GPU cost over the course of pretraining.

6 Zero-Shot Benchmark Evaluation

To assess the capabilities acquired by AgentGate-Base-142M during pretraining, we evaluated the model on three established language understanding benchmarks: ARC-Easy, HellaSwag, and PIQA.

We additionally compared AgentGate against three publicly available pretrained language models of comparable parameter scale: GPT-2 Small (124M), Pythia-160M, and SmolLM-135M. These comparisons provide context for interpreting AgentGate's performance relative to both earlier and more recent pretrained Transformer models.

6.1 Evaluation Setup

All evaluations were conducted using EleutherAI's LM Evaluation Harness (version 0.4.13) under a zero-shot configuration, with no task-specific fine-tuning or in-context demonstrations. We evaluated the complete benchmark splits without limiting the number of examples.

The selected benchmarks were:

- **ARC-Easy:** A multiple-choice question-answering benchmark containing 2,376 test examples. It evaluates general scientific knowledge and elementary reasoning.
- **HellaSwag:** A commonsense reasoning benchmark containing 10,042 validation examples. Given a context, the model must identify the most plausible continuation among four candidates.
- **PIQA:** A physical commonsense reasoning benchmark containing 1,838 validation examples. Each question presents two candidate solutions to a practical, everyday problem.

For each example, the evaluation framework computes the conditional log-likelihood of each candidate continuation given the question or context. The predicted answer is determined by selecting the candidate with the highest score.

We report two evaluation metrics:

- **Accuracy (Acc):** The proportion of examples for which the candidate with the highest total conditional log-likelihood is correct.
- **Normalized Accuracy (Acc_norm):** Accuracy obtained after normalizing each candidate's log-likelihood by its length, following the evaluation harness's task configuration.

Length normalization reduces the influence of candidate length on likelihood-based answer selection. Both metrics are reported because normalization can alter candidate rankings and does not necessarily improve accuracy across all tasks.

Since AgentGate uses a custom model architecture and byte-level BPE tokenizer, we integrated it with the evaluation harness through a custom model adapter. The adapter performs teacher-forced inference and extracts the log-likelihoods of candidate continuation tokens. The baseline models were evaluated through the harness's Hugging Face model interface.

Table 4: Zero-shot benchmark comparison of pretrained language models. Acc denotes raw accuracy, while Acc_{norm} denotes length-normalized accuracy. All values are percentages (%); higher is better. Bold values indicate the best result in each column.

Model	Params	ARC-Easy		HellaSwag		PIQA	
		Acc	Acc_{norm}	Acc	Acc_{norm}	Acc	Acc_{norm}
GPT-2 Small	124M	43.81	39.48	28.92	31.14	62.89	62.51
Pythia-160M	160M	43.64	39.65	28.37	30.27	62.30	61.92
AgentGate (Ours)	142.4M	50.46	45.41	29.14	32.03	62.95	62.40
SmolLM-135M	135M	**61.62**	**56.48**	**35.18**	**42.60**	**68.61**	**68.12**

6.2 Comparison Models

We selected three pretrained language models with parameter counts close to AgentGate's 142.4 million parameters.

- **GPT-2 Small (124M):** An earlier decoder-only Transformer pretrained on the WebText corpus, containing approximately 40 GB of text (roughly 10 billion tokens). It serves as a widely recognized historical baseline.
- **Pythia-160M:** A pretrained decoder-only language model from the Pythia suite, trained on approximately 300 billion tokens from The Pile dataset. We evaluated its final pretrained checkpoint.
- **SmolLM-135M:** A modern small language model employing rotary positional embeddings (RoPE), RMSNorm, and SwiGLU feed-forward layers. It was pretrained on approximately 600 billion tokens from the SmolLM-Corpus.

These models offer reference points at a similar parameter scale, although they differ in architecture, tokenizer, training corpus, and pretraining budget.

6.3 Benchmark Results

Table 4 summarizes the zero-shot evaluation results. All values are reported as percentages, with higher scores indicating better performance.

AgentGate achieved raw accuracies of 50.46%, 29.14%, and 62.95% on ARC-Easy, HellaSwag, and PIQA, respectively. Its corresponding length-normalized accuracies were 45.41%, 32.03%, and 62.40%.

On ARC-Easy, AgentGate outperformed both GPT-2 Small and Pythia-160M. In terms of normalized accuracy, AgentGate achieved improvements of 5.93 and 5.76 percentage points over GPT-2 Small and Pythia-160M, respectively. This was the largest observed advantage over the two historical baselines.

On HellaSwag, AgentGate achieved a normalized accuracy of 32.03%, compared with 31.14% for GPT-2 Small and 30.27% for Pythia-160M. Although AgentGate obtained the highest score among these three models, the differences were relatively small.

For PIQA, the three models exhibited similar performance. AgentGate obtained a normalized accuracy of 62.40%, compared with 62.51% for GPT-2 Small and 61.92% for Pythia-160M. These differences are too small to support a meaningful performance advantage.

SmolLM-135M outperformed AgentGate on all three benchmarks. The normalized accuracy gaps were 11.07, 10.57, and 5.72 percentage points on ARC-Easy, HellaSwag, and PIQA, respectively.

6.4 Discussion

The benchmark results demonstrate that AgentGate acquired measurable language understanding and commonsense capabilities through pretraining on approximately 2.7 billion tokens.

Despite its relatively limited pretraining budget, AgentGate achieved higher normalized accuracy than Pythia-160M across all three evaluated benchmarks and surpassed GPT-2 Small on ARC-Easy and HellaSwag. Its strongest relative performance was observed on ARC-Easy, suggesting that the model acquired useful scientific and factual associations from its pretraining corpus.

The comparison with SmolLM-135M reveals a substantial remaining performance gap. Although AgentGate and SmolLM have similar parameter counts and share several modern architectural components, SmolLM was pretrained

on a considerably larger token budget. AgentGate therefore cannot be expected to match its performance solely on the basis of architectural similarity.

7 Mechanistic Interpretability: Investigating Induction Circuits

While the benchmark results provide an external assessment of AgentGate's capabilities, they offer limited insight into the internal computations responsible for its predictions. Beyond evaluating what the model has learned, we are also interested in understanding how it processes and utilizes information within its context. To investigate this, we conduct a small-scale mechanistic interpretability study focusing on sequence copying behavior and the attention head interactions associated with induction circuits. The following section presents this analysis.

7.1 Introduction

AgentGate is a 142-million-parameter decoder-only Transformer pretrained on approximately 2.7 billion tokens. After pretraining, we wanted to examine what the model had learned beyond its next-token prediction accuracy. In particular, could it recognize a sequence that had appeared earlier in its context and use that sequence to predict the next token?

This is related to induction heads, a pattern studied in Transformer interpretability research [7]. An induction head may help a model identify a previous occurrence of the current token and attend to the token that followed it. Such behavior offers a simple example of learning from the current context without changing the model weights.

Consider a context containing "A B C D E" followed later by "A B C D". The next token could be E because E followed the same prefix earlier.

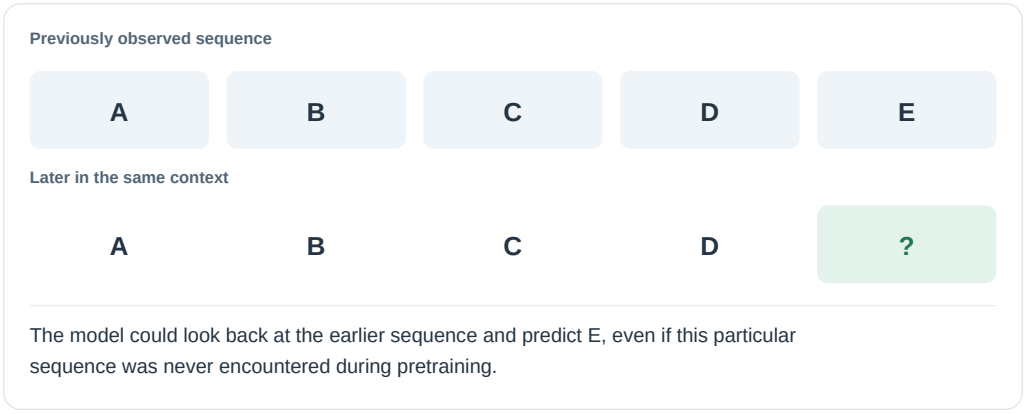


Figure 7: Conceptual example of in-context pattern copying.

We investigate this phenomenon in four stages. First, we test whether exact sequence repetition improves prediction. Second, we search for attention heads with patterns expected from induction circuits [8, 7]. Third, we disable selected heads to test whether they matter for copying. Finally, we ask whether earlier previous-token heads influence later induction-pattern heads.

8 Model and Evaluation Setting

AgentGate is a 142-million-parameter decoder-only Transformer with 24 layers and 10 attention heads per layer, giving a total of 240 attention heads. Its hidden size is 640, and each attention head has a dimension of 64.

We used the pretrained checkpoint in evaluation mode without modifying its learned weights. To inspect individual attention heads, we implemented a manual attention computation path that exposes their attention weights. Before conducting the experiments, we verified that this implementation produced nearly identical outputs to PyTorch's optimized scaled dot-product attention (SDPA) implementation.

The experiments used synthetic token sequences constructed as:

```
[prefix] + [random block] + [second block]
```

The second block was either an exact copy or a shuffled version of the first block, depending on the evaluation condition.

9 Experiment 1: Repeated-sequence behavior

9.1 Question and motivation

Before searching for induction heads, we first asked a simpler question: can AgentGate use an exact repetition at all? If it has learned to exploit a sequence already in context, predicting that sequence a second time should be easier than predicting its initial occurrence. We also wanted to check whether merely seeing the same token identities was enough.

9.2 Experimental setup

We evaluated three conditions. In the first-copy condition, the model predicts a randomly sampled token block during its first occurrence. In the exact second-copy condition, the same block appears again in its original order. In the shuffled second-copy condition, the original block is followed by a random permutation of its tokens. The shuffled sequence contains the same tokens but disrupts most of their original transitions, although some may remain intact by chance.

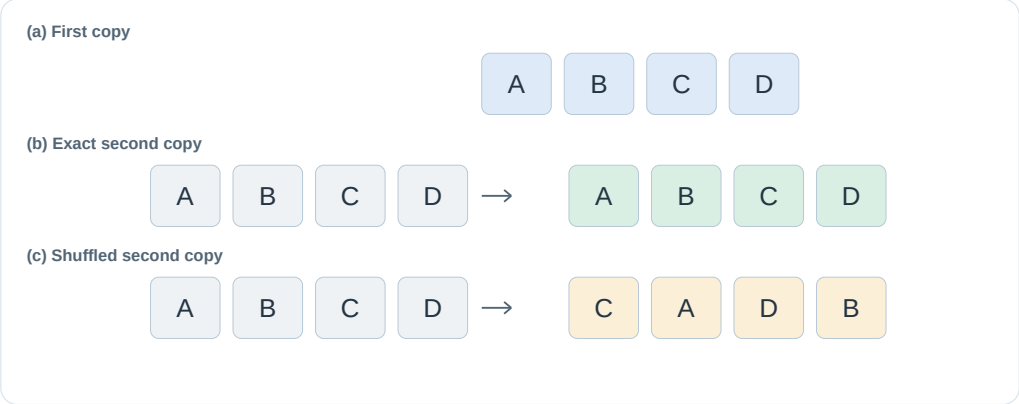


Figure 8: Three evaluation conditions

We tested three block lengths: 32, 64, and 128 tokens. For each length, we generated 100 random examples and evaluated all three conditions.

To measure prediction quality, we used next-token negative log-likelihood (NLL):

$$\mathrm{NLL} = -\log P(\text{correct next token} \mid \text{preceding context}) \tag{61}$$

This measures how well the model predicts each token using only the tokens preceding it. A lower NLL indicates that the model assigns a higher probability to the correct next token.

Calculating the position-wise NLL. For each example, we recorded the NLL at every token position within the evaluated block. We then averaged the NLL at each position across the 100 examples. For instance, at block length 64, the NLL values at position 10 were averaged across all 100 examples to obtain a single value for that position. Repeating this for every position produced the three curves shown in Figure 9: first copy, exact second copy, and shuffled second copy. We followed the same procedure for block lengths 32 and 128.

Calculating the overall NLL. To summarize each condition with a single value, we averaged its position-wise NLL values across the block, excluding the first position. Let L_{first} and L_{exact} denote the resulting mean per-token NLL for the first copy and exact second copy, respectively.

We define the copy advantage as:

$$A = L_{\text{first}} - L_{\text{exact}} \tag{62}$$

A positive copy advantage means that the model predicts the exact second copy more accurately than the first copy. A larger value indicates a greater improvement from sequence repetition.

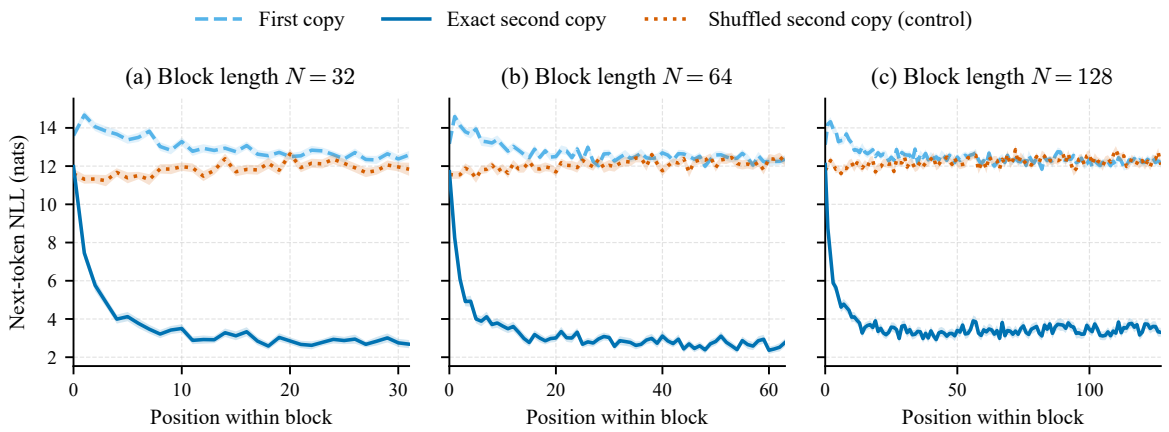


Figure 9: Position-wise NLL for the three evaluation conditions across block lengths of 32, 64, and 128 tokens.

9.3 Results and Interpretation

Figure 9 shows the position-wise NLL for the three evaluation conditions across block lengths of 32, 64, and 128 tokens.

A. Exact sequence repetition improves prediction

Across all three block lengths, the exact second copy has substantially lower NLL than the first copy. The copy advantage ranges from approximately 8.90 to 9.61 nats.

An interesting observation is that the exact second-copy loss drops sharply after the first few positions. This suggests that as the model encounters more tokens from the repeated block, it becomes better at predicting the remaining sequence. Such behavior is consistent with the model recognizing a matching sequence in its earlier context and using it to predict subsequent tokens.

B. Sequence order matters

The shuffled second copy remains considerably harder to predict than the exact second copy, even though both contain the same tokens.

For example, at block length 64, the exact second copy has an average NLL of 3.14, compared with 12.06 for the shuffled second copy.

This suggests that the improvement is not simply due to the model having encountered the same tokens before. Instead, preserving their original order appears to play an important role in the model's ability to exploit repeated contextual information.

The shuffled second copy also performs slightly better than the first copy in some cases. This may be due to prior exposure to the same tokens. However, we have not separately investigated this possibility.

C. Effect of block length

As the block length increases, the first-copy and shuffled second-copy loss curves show some overlap. The reason for this behavior remains unclear and requires further investigation.

Despite this variation, the exact second copy consistently achieves much lower prediction loss across all three block lengths.

What do these results tell us? Our first experiment establishes that AgentGate can exploit exact sequence repetition to improve next-token prediction, even when the repeated sequences consist of randomly sampled tokens. Since the model's weights remain unchanged, this improvement comes from information available within the current context.

However, these results only describe what the model can do, not how it performs this computation.

This leads to our next question: Which internal components allow AgentGate to recognize a repeated sequence and predict its continuation?

Prior work [7] on induction heads suggests that certain attention heads may perform this kind of computation. We therefore turn to AgentGate's 240 attention heads to examine whether any exhibit the attention patterns associated with induction circuits.

10 Experiment 2: Attention-head specialization

10.1 What patterns are we looking for?

The previous experiment showed that AgentGate can exploit repeated sequences, but it did not explain how the model achieves this behavior. Prior works [7, 8] on induction heads suggests that sequence copying may involve two complementary types of attention heads: previous-token heads and induction heads.

Figure 10 illustrates how these two types of heads can work together.

A. Previous-token attention

As shown in Figure 10(a), consider the sequence A B C D. When processing D, a previous-token head attends to C, the immediately preceding token. This can make information about C available in the representation at the D position.

B. Induction-pattern attention

Now consider the sequence A B C D ... A B C, shown in Figure 10(b). When the model encounters the second C, the expected next token is D, since D followed C in the earlier sequence.

An induction head at the second C may attend to the earlier D position. In a possible two-stage mechanism, the previous-token head helps associate the earlier D with its preceding C. The induction head can then use this association to locate the earlier D when C appears again, helping the model predict the next token.

Our objective in this experiment was to identify whether any attention heads in AgentGate exhibit these two patterns.

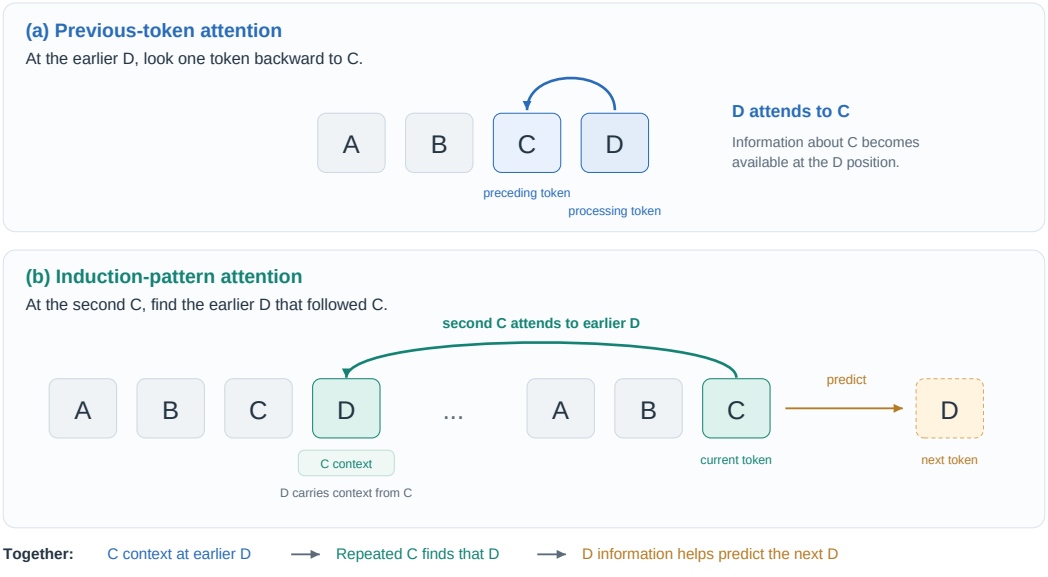


Figure 10: Conceptual two-stage attention pattern

10.2 How we measured head specialization

AgentGate contains 24 Transformer layers, each with 10 attention heads, giving a total of 240 heads. We evaluated all 240 heads using 100 synthetic inputs, each consisting of a prefix token followed by a 64-token block and its exact second copy. Each input therefore contained 129 tokens.

For every head, we extracted its attention matrix of size 129×129, where $A[t, j]$ represents the attention weight from position t to position j.

We calculated two scores corresponding to the patterns illustrated in Figure 10.

A. Previous-token attention score

To measure the pattern in Figure 10(a), we recorded how much attention each head assigned to the immediately preceding position, $t - 1$. We then averaged these weights across the evaluated positions and examples:

$$S_{\text{prev}} = \frac{1}{|Q_{\text{prev}}|} \sum_{t \in Q_{\text{prev}}} A[t, t-1] \tag{63}$$

Here, Q_{prev} represents the evaluated query positions. For example, a score of 0.90 means that the head assigns, on average, 90% of its attention to the immediately preceding position.

B. Induction-pattern attention score

To measure the pattern in Figure 10(b), we examined how much attention each head assigned to the expected earlier continuation position.

For a repeated block of length N, this position is $t - N + 1$, where t is the current position in the exact second copy. We calculated:

$$S_{\text{ind}} = \frac{1}{|Q_{\text{ind}}|} \sum_{t \in Q_{\text{ind}}} A[t, t-N+1] \tag{64}$$

Here, Q_{ind} contains the valid query positions in the exact second copy. For example, an induction-pattern score of 0.87 means that the head assigns, on average, 87% of its attention to the expected earlier continuation position.

Both scores were averaged across the 100 examples. A high score indicates that a head strongly exhibits the corresponding attention pattern. However, these scores only measure where a head attends, not whether it contributes to the model's predictions.

10.3 What we found

Figure 11 presents the previous-token and induction-pattern attention scores for all 240 attention heads. Each row represents a Transformer layer, while each column represents an attention head. Brighter cells indicate higher attention scores, meaning that the head assigns more attention to the position we are measuring.

A. Previous-token attention

As shown in Figure 11(a), the strongest previous-token attention scores are concentrated around Layers 15 and 16. In particular, L16H2 (Layer 16, Head 2, using zero-based indexing) achieves a score of 0.900, followed by L16H4 with 0.836 and L15H2 with 0.709. For example, L16H2's score of 0.900 means that it assigns approximately 90% of its attention to the immediately preceding position, averaged across our evaluation examples. This suggests that some heads have developed a strong preference for looking one token backward.

B. Induction-pattern attention

Figure 11(b) shows a different pattern. The strongest induction scores appear mainly in Layers 18 to 20. The most notable heads are L19H8 (0.867), L19H9 (0.853), and L18H8 (0.631).

Recall our example: A B C D ... A B C.

When processing the second C, an induction-pattern head is expected to attend to the earlier D, since D followed C in the original sequence. L19H8's score of 0.867 means that, on average, approximately 86.7% of its attention is assigned to such earlier continuation positions. This makes L19H8 a strong candidate for induction-like behavior.

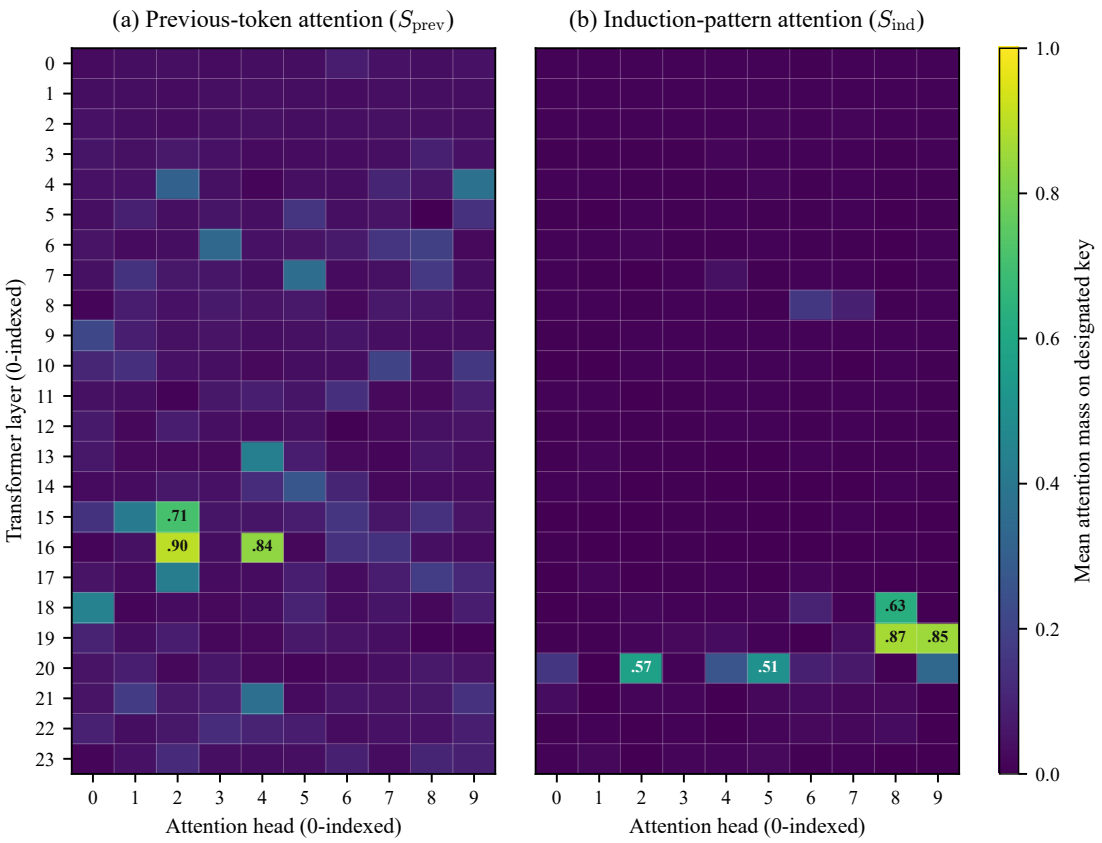


Figure 11: Attention-head specialization across AgentGate's 24 Transformer layers

C. What do these findings suggest?

An interesting observation emerges when we compare the two heatmaps. The strongest previous-token heads appear in earlier layers, particularly Layer 16, while the strongest induction-pattern heads appear in later layers, particularly Layer 19.

This ordering is consistent with the two-stage mechanism discussed in Section 4.1. Earlier heads may help associate a token with its preceding context, while later heads may use that information to locate a matching continuation.

However, attention patterns alone cannot establish whether these heads actually contribute to copying. A head may attend strongly to the expected position without passing useful information to the model's final prediction.

So far, we have identified attention heads that exhibit the patterns expected from an induction circuit. But this raises an important question: Are these heads actually responsible for AgentGate's improved prediction of the exact second copy, or do they simply exhibit attention patterns associated with copying?

To investigate this, our next experiment temporarily disables the selected heads and measures how their removal affects prediction performance.

11 Experiment 3: Causal Attention-Head Ablation

11.1 Why Are Attention Scores Not Enough?

In Experiment 2, we identified several attention heads that strongly exhibit previous-token and induction-pattern attention. However, a high attention score does not necessarily mean that a head contributes useful information to the model's predictions.

For example, L19H8 assigns most of its attention to the expected earlier continuation position. But what if the information retrieved by this head has little influence on the final prediction? Its induction-pattern score would still be high.

To determine whether these heads actually contribute to sequence copying, we need to go beyond observing their attention patterns [9]. Our objective is to test whether disabling the selected heads reduces AgentGate's ability to predict the exact second copy.

11.2 How We Performed the Interventions

We generated 100 new repeated-sequence examples with a block length of 64 tokens. First, we evaluated the normal model with all attention heads active. The average NLL was approximately 12.65 for the first copy and 3.19 for the exact second copy, giving a baseline copy advantage of 9.46 nats.

Next, we temporarily disabled selected attention heads, one at a time or in pairs, and measured how their removal affected prediction performance.

Each attention head in AgentGate produces a 64-dimensional output vector. To disable a head, we replaced this vector with zeros before the multi-head attention output projection. This removes the selected head's contribution while leaving the other heads active. The intervention was performed only during inference, without modifying the model's learned weights.

However, disabling any component of a pretrained model could potentially affect its performance. Therefore, we also selected comparison heads from the same layers that showed relatively low previous-token or induction-pattern attention scores.

For example, L19H8 is a strong induction-head candidate, while L19H0 and L19H3 have low induction-pattern scores. If disabling all three heads produced similar performance degradation, we would have little evidence that L19H8 is particularly important for copying. Conversely, if disabling L19H8 caused a much larger effect, it would support its specific contribution to the behavior.

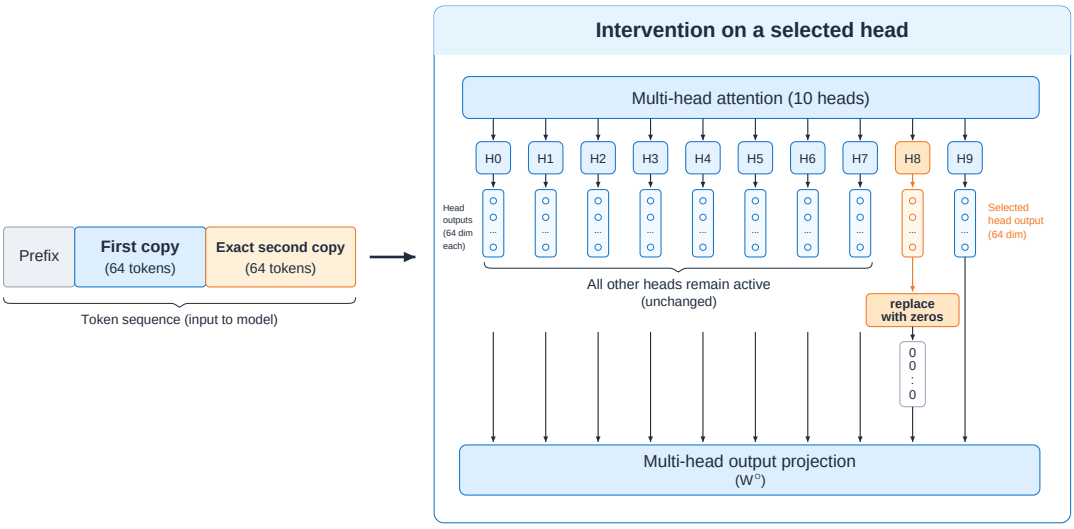


Figure 12: Intervention on a selected head.

We performed the following interventions:

For each intervention, we evaluated the same 100 examples and measured the average NLL for both the first copy and the exact second copy.

Measuring the effect of ablation Recall that the copy advantage is defined as:

$$A = L_{\text{first}} - L_{\text{exact}} \tag{65}$$

We measured how much this advantage decreased after disabling a head:

$$\Delta A = A_{\text{normal}} - A_{\text{ablated}} \tag{66}$$

Table 5: Overview of Attention-Head Interventions

Category	Heads tested	Purpose
Induction-head candidates	L19H8, L19H9, L18H8	Disable each head individually to test its contribution to copying.
Combined induction-head ablation	L19H8 + L19H9	Test the effect of disabling two strong induction-head candidates together.
Previous-token candidates	L16H2, L16H4	Disable each head individually to examine its contribution to copying.
Combined previous-token ablation	L16H2 + L16H4	Determine whether disabling both heads has a stronger effect than disabling either alone.
Comparison heads	L19H0, L19H3, L18H1, L16H0, L16H7	Check whether the observed performance degradation is specific to the selected candidates rather than a general effect of removing attention heads.

Expanding this expression gives:

$$\Delta A = (L_{\text{exact,ablated}} - L_{\text{exact,normal}}) - (L_{\text{first,ablated}} - L_{\text{first,normal}}) \tag{67}$$

Here, L_{first} and L_{exact} represent the average next-token NLL for the first and exact second copies, respectively.

A positive ΔA means that disabling the selected head reduces the model's copy advantage. In other words, the intervention harms prediction of the exact second copy more than prediction of the first copy.

For example, suppose disabling a head increases first-copy NLL by 0.1 nats and exact second-copy NLL by 2.3 nats. Then:

$$\Delta A = 2.3 - 0.1 = 2.2 \text{ nats} \tag{68}$$

This indicates that the ablation disproportionately affects the model's ability to benefit from sequence repetition.

By comparing ΔA across candidate and comparison heads, we can determine whether the attention heads identified in Experiment 2 make a meaningful causal contribution to copying.

11.3 Results and interpretation

Figure 13 shows how disabling different attention heads affects AgentGate's copy advantage. Bars extending to the right indicate a reduction in copy advantage, meaning that the model becomes relatively worse at predicting the exact second copy. We compare the effects of induction-head candidates, previous-token candidates, and comparison heads.

A. Induction-head candidates

As shown in Figure 13, disabling L19H8 reduces the copy advantage by approximately 2.33 nats, while disabling L19H9 reduces it by 1.20 nats. When both heads are disabled simultaneously, the reduction increases to approximately 4.04 nats.

This indicates that both heads contribute to the model's ability to exploit repeated sequences. Interestingly, disabling them together produces a larger reduction than the sum of their individual effects, suggesting that their contributions may interact.

B. Previous-token candidates

The earlier previous-token heads show a similar pattern. Disabling L16H2 or L16H4 individually reduces the copy advantage by approximately 1.41 and 1.37 nats, respectively. However, disabling both heads produces a much larger reduction of approximately 5.31 nats, as shown in Figure 13.

To understand this effect, consider the prediction losses. When both heads are disabled, the first-copy NLL increases only slightly, from 12.65 to 12.75. In contrast, the exact second-copy NLL increases substantially, from 3.19 to 8.61.

This difference is important. If the intervention simply made the model worse at predicting tokens in general, we might expect both conditions to be affected similarly. Instead, the exact second copy suffers much more, suggesting that these previous-token heads play an important role in exploiting repeated contextual information.

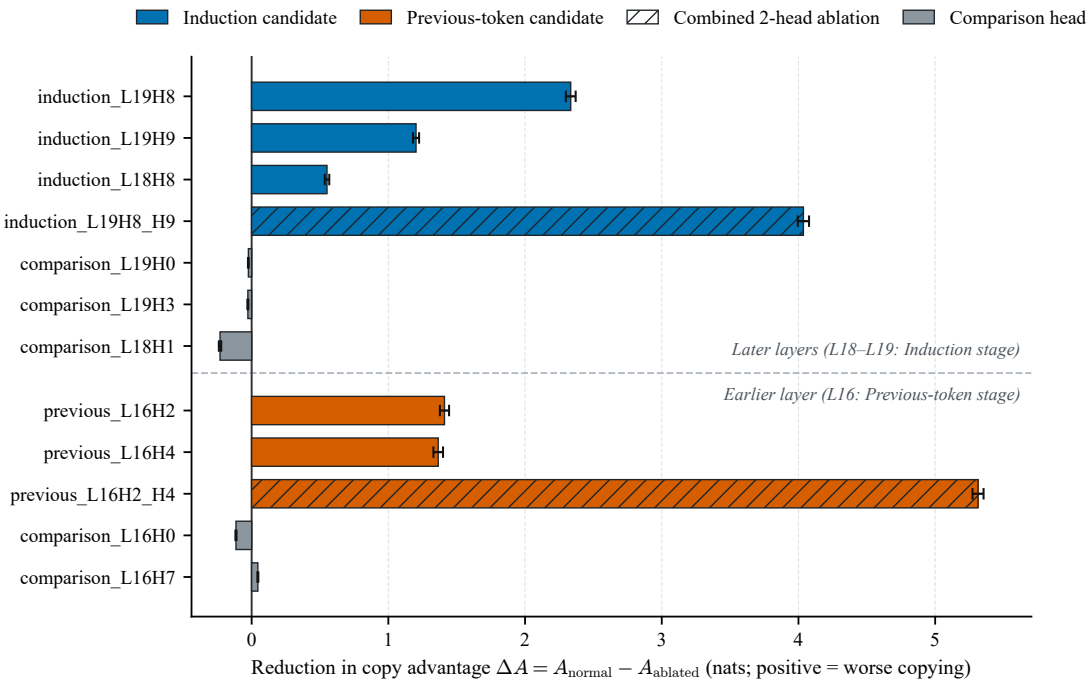


Figure 13: Copy advantage after head ablation.

C. Comparison heads

Finally, Figure 13 shows that disabling the selected comparison heads produces much smaller changes in copy advantage, with most values close to zero.

This strengthens our findings. If disabling any attention head caused a similar reduction, we could not confidently associate the observed effects with our selected candidates. Instead, the much larger effects of L19H8, L19H9, L16H2, and L16H4 support their specific causal contributions to the measured copying behavior.

What do these results tell us? We have now moved beyond identifying interesting attention patterns. The ablation results show that the selected induction-head and previous-token candidates are not merely attending to particular positions. Their outputs also contribute to AgentGate's copying performance.

However, one important question remains unanswered. We know that the previous-token heads in Layer 16 and the induction-head candidates in Layer 19 both contribute to copying, but do the earlier heads actually influence how the later heads attend?

If these heads participate in the two-stage mechanism proposed earlier, disabling the previous-token heads should also disrupt the induction-attention patterns of the later heads. This motivates our next experiment, where we investigate the functional dependency between them.

12 Experiment 4: Functional Dependency Between Earlier and Later Heads

12.1 The Question Left by Ablation

In Experiment 3, we found that disabling either previous-token heads or induction-head candidates significantly reduced AgentGate's copying performance. However, this does not necessarily mean that the two groups of heads work together.

Recall our earlier example:

A B C D ... A B C

In the proposed two-stage mechanism, an earlier previous-token head attends from the first D to its preceding C, potentially making information about C available at the D position. Later, an induction-pattern head at the second C may use this information to attend to the earlier D, helping the model predict the next token.

If these two groups of heads are functionally connected, disabling the earlier previous-token heads should disrupt the induction-attention patterns of the later heads.

This is what we investigate in Experiment 4. Unlike the previous experiment, where we measured changes in prediction loss, we now examine how disabling earlier heads affects the attention scores of later heads. Importantly, the later heads themselves remain active throughout the intervention.

12.2 How We Tested the Dependency

We evaluated 100 repeated-sequence examples with a block length of 64 tokens, following the same input structure as the previous experiments.

First, we ran the normal model and measured the induction-pattern attention scores of three later heads: L19H8, L19H9, and L18H8. These scores served as our baseline.

Next, we disabled the earlier previous-token heads L16H2 and L16H4, first individually and then together. After each intervention, we measured the induction scores of the same later heads again.

For example, when disabling L16H2, we did not modify L19H8 itself. Instead, we examined whether removing L16H2's output changed how strongly L19H8 attended to the expected earlier continuation position.

We also performed similar interventions on comparison heads from Layer 16 and nearby layers. This allowed us to check whether the downstream changes were specific to our selected previous-token heads rather than a general consequence of disabling earlier attention heads.

Measuring the change in induction score Recall that the induction-pattern attention score, S_{ind}, measures how much attention a head assigns to the expected earlier continuation position.

We calculated the change in this score as:

$$\Delta S = S_{\text{ablated}} - S_{\text{normal}} \tag{69}$$

Here, S_{normal} is the induction score of a selected later head when all heads are active, while S_{ablated} is its score after disabling the selected earlier heads.

A negative ΔS means that the later head assigns less attention to the expected earlier continuation position after the intervention. If this reduction is substantially larger for our previous-token candidates than for the comparison heads, it provides evidence that the earlier heads functionally influence the later induction-attention patterns.

12.3 Results and Interpretation

Figure 14 shows how disabling earlier attention heads affects the induction-pattern attention scores of three later heads: L19H8, L19H9, and L18H8. Bars extending to the left indicate a decrease in induction score, meaning that the later head assigns less attention to the expected earlier continuation position.

A. Individual previous-token head ablations

As shown in Figure 14, disabling L16H2 reduces the induction score of L19H8 from 0.903 to 0.826. Similarly, disabling L16H4 reduces it to 0.801.

Although both interventions weaken L19H8's induction-pattern attention, the reductions are relatively modest. The head still assigns most of its attention to the expected earlier continuation position when either previous-token head is disabled individually.

B. Combined previous-token head ablation

The effect becomes much stronger when L16H2 and L16H4 are disabled simultaneously. The induction score of L19H8 drops from 0.903 to 0.376, a relative reduction of approximately 58%. Similar reductions occur for L19H9 (0.892 to 0.345) and L18H8 (0.665 to 0.150).

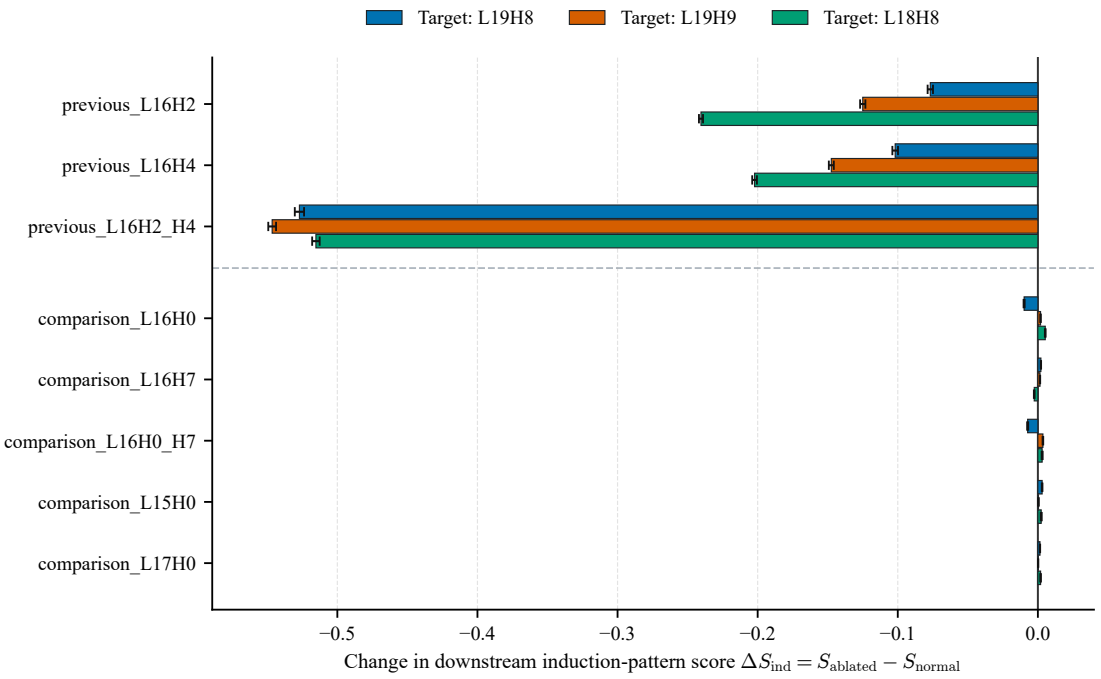


Figure 14: Change in later induction-pattern attention following earlier-head interventions.

Notably, the combined intervention produces a much larger reduction than either individual intervention. This suggests that the two earlier heads may provide overlapping or complementary information that supports later induction-pattern attention.

C. Comparison-head ablations

Figure 14 also shows that disabling the selected comparison heads produces very small changes in the downstream induction scores. For example, disabling L16H0 and L16H7 together changes L19H8's score by only approximately -0.007, compared with -0.527 when L16H2 and L16H4 are disabled together. It suggests that the downstream disruption is not simply caused by disabling arbitrary earlier attention heads. Instead, the selected previous-token heads have a more specific influence on the induction-attention patterns of later heads.

What do these results tell us? Our objective was to determine whether earlier previous token heads functionally influence later induction pattern heads. The results provide strong evidence that they do.

Disabling L16H2 and L16H4 substantially reduces the induction scores of L19H8, L19H9, and L18H8, while disabling comparison heads has little effect. This demonstrates a causal dependency between the selected earlier heads and the induction-attention behavior of later heads.

Combined with Experiment 3, these findings support the proposed two-stage copying mechanism. We first established that both groups of heads contribute to sequence copying performance. We then demonstrated that disrupting the earlier previous token heads weakens the induction attention patterns of the later heads.

Therefore, our experiments provide evidence that AgentGate's copying behavior is supported by interacting attention head computations across different Transformer layers, consistent with an induction-like circuit.

The precise information pathways between these components remain an interesting direction for future investigation.

References

[1] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[2] Modal Labs. Modal: Cloud functions and distributed computing platform. https://modal.com/.

[3] Jessica Rumbelow and Matthew Watkins. SolidGoldMagikarp (plus, prompt generation). LessWrong, February 2023. https://www.lesswrong.com/posts/aPeJE8bSo6rAFoLqg.

[4] Jackson Petty, Sjoerd van Steenkiste, Ishita Dasgupta, Fei Sha, Dan Garrette, and Tal Linzen. The impact of depth on compositional generalization in transformer language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7239–7252, 2024.

[5] Sho Takase, Shun Kiyono, Sosuke Kobayashi, and Jun Suzuki. B2T connection: Serving stability and performance in deep transformers. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 3078–3095, Toronto, Canada, July 2023. Association for Computational Linguistics.

[6] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. On layer normalization in the transformer architecture. In *International conference on machine learning*, pages 10524–10533. PMLR, 2020.

[7] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html.

[8] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. https://transformer-circuits.pub/2021/framework/index.html.

[9] Kevin Ro Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations (ICLR)*, 2023.